

GitLab Day-1

Architecture & Complete Feature Guide

DevOps Shack | devopsshack.com | [@devopsshack](https://twitter.com/devopsshack)

Free	Premium	Ultimate
For individuals working on personal projects and open source contributions	For scaling organizations seeking enhanced productivity and collaboration	For enterprises requiring advanced security and compliance capabilities
\$0 per user/month, No credit card required		Contact us for custom pricing
Get started	Buy Premium Try for free	Try for free
Features include: ✓ Bring your own storage and runners	Everything from Free, plus: ✓ Advanced CI/CD ✓ Team Project Management ✓ SLA Management ✓ Priority Support GitLab Duo Agent Platform ✓ Includes \$12 in GitLab Credits per user per month* Paid Add-ons ✓ Additional GitLab Credits Learn more about Premium >	Everything from Premium, plus: ✓ Application Security Testing ✓ Software Supply Chain Security ✓ Vulnerability Management ✓ Strategic Portfolio Management ✓ Value Stream Management ✓ Compliance and Governance GitLab Duo Agent Platform ✓ Includes \$24 in GitLab Credits per user per month* Paid Add-ons ✓ Additional GitLab Credits ✓ Enterprise Agile Planning Seats Learn more about Ultimate >

Introduction to GitLab

GitLab is an all-in-one DevOps platform that combines source code management, CI/CD pipelines, security scanning, project management, monitoring, and analytics under a single web interface. Unlike GitHub, which focuses on code hosting, GitLab provides the full software delivery lifecycle in one tool.

GitLab vs GitHub — Key Difference	Details
Scope	GitLab: full DevOps platform. GitHub: primarily code + Actions CI.
Built-in CI/CD	GitLab has native pipelines; GitHub Actions is bolt-on.
Security Scanning	GitLab includes SAST, DAST, container scanning out of the box.
Project Management	GitLab includes Issues, Epics, Milestones, Boards natively.
Self-hosting	GitLab Community Edition is free to self-host; GitHub Enterprise costs more.

Key Idea

Unlike traditional tools where you combine:

- GitHub (code)
- Jenkins (CI)
- Nexus (artifact storage)
- Jira (planning)

GitLab combines everything into **one unified platform**.

Why DevOps/SRE Teams Use GitLab

- Single source of truth
- Tight CI/CD integration
- Built-in DevSecOps
- Better traceability (code → pipeline → deployment)

2. GitLab vs Other Tools

First, Understand the Core Difference

Before comparing, let's understand

- **Git** = Version Control System (tool)
- **GitLab** = Complete DevOps Platform
- Others (GitHub, Gitea, Jenkins) = Partial solutions

Git vs GitLab

Git

- Distributed version control system
- Works locally
- No UI, no collaboration platform
- No CI/CD, no issue tracking

GitLab

- Uses Git internally
- Adds:
 - Web UI
 - CI/CD pipelines
 - Issue tracking
 - Security scanning
 - DevOps lifecycle tools

Key Difference

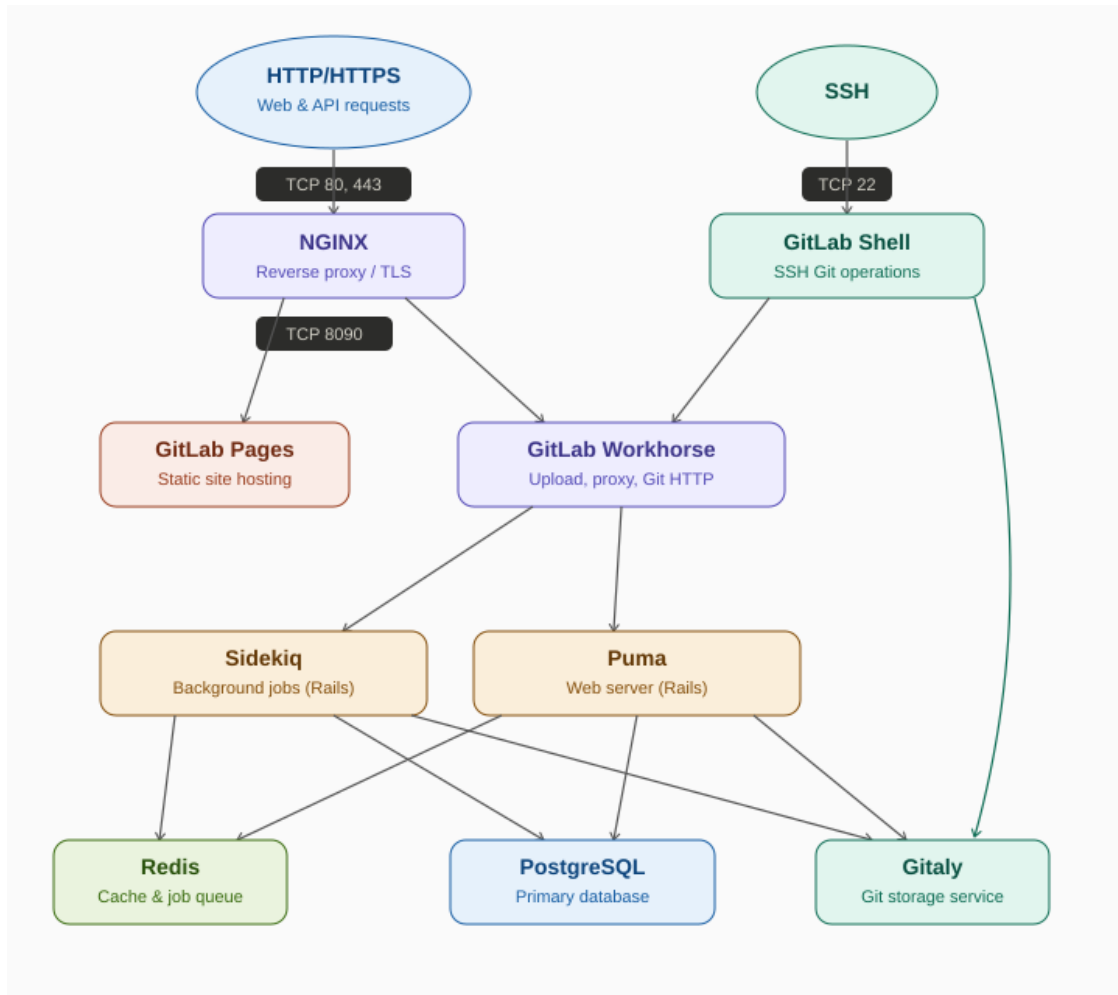
Git is a **tool**, GitLab is a **platform built around Git**

DevOps Insight

GitHub is still **developer-first**, GitLab is **DevOps-first**

The Big Picture

GitLab is not one application — it is 9 specialised services working together. Each one does exactly one job and passes work to the next.



Entry

HTTP/HTTPS · SSH

Routing

NGINX · Shell

Application

Workhorse · Puma · Sidekiq

Storage

Redis · PostgreSQL · Gitaly

The 9 Components

Each component explained in two things: what it is, and what it does.

1. GitLab Rails Application

- Main web application
- Handles:
 - UI
 - API
 - Authentication
 - Business logic

LAYER 1 — ENTRY • How requests reach GitLab

	HTTP / HTTPS Port 80 & 443	The standard web protocol. Your browser, API tools, and git over HTTPS all use this port to reach GitLab.
	SSH Port 22	A secure tunnel for developers. Used only for git push and git pull when authenticated with an SSH key. Faster than HTTPS for daily coding.

LAYER 2 — ROUTING • Directs requests to the right service

	NGINX Web proxy	The first thing that receives your browser request. It handles TLS encryption, blocks bad traffic, serves static files (images, CSS), and forwards everything else to Workhorse.
	GitLab Shell SSH handler	Receives all SSH connections. Checks your SSH key, confirms you have permission to access that repo, then hands the Git operation off to Gitaly.

LAYER 3 — APPLICATION • Where the logic runs

💡 What is the Rails App?

Ruby on Rails (commonly called "Rails") is a web framework written in the Ruby programming language. Think of it as the brain of GitLab — it contains all the business logic: creating issues, managing users, checking permissions, running pipelines, rendering pages.

Puma and Sidekiq are not two different apps. They are two different ways of running the same Rails code — Puma handles requests right now (real-time), Sidekiq handles tasks in the background (async).

	Workhorse Go proxy	Sits between NGINX and the Rails app. Handles large file uploads, artifact downloads, and git over HTTPS — so these heavy tasks never slow down the main application.
	Puma Runs Rails — real-time	Puma is the web server that runs the Rails app for live requests. Every time you load a page, call the API, or submit a form — Puma handles it and sends back a response immediately.
	Sidekiq Runs Rails — background	Sidekiq also runs the Rails app, but for tasks that don't need an instant response. After your code push, Sidekiq starts the CI pipeline, sends notification emails, and delivers webhooks — all in the background while you keep working. • Background job processor • Handles: ◦ Emails ◦ Pipeline processing ◦ Async tasks

LAYER 4 — STORAGE • Where all data lives permanently

	Redis In-memory store	Stores data in RAM — making it extremely fast. GitLab uses it for login sessions, page cache, and as the job queue between Puma (writes jobs) and Sidekiq (reads jobs).
	PostgreSQL Main database	Permanent, structured storage. Holds users, projects, issues, MRs, CI pipeline records, permissions, and audit logs. Everything GitLab needs to remember. Stores: ◦ User data ◦ Metadata ◦ Issues, pipelines Important: GitLab does NOT store Git repositories in PostgreSQL.
	Gitaly Git storage service	The only component that can read or write repository files. Every branch, commit, and file change lives here. Nothing bypasses Gitaly — all repo access goes through its API. Service responsible for Git operations

- Handles:
 - Clone
 - Fetch
 - Push

Important for scaling Git operations.

How It All Connects

You open a GitLab page in your browser

1	Browser	Sends HTTPS request to GitLab on port 443
2	NGINX	Decrypts TLS, passes request to Workhorse
3	Workhorse	Not a heavy request — passes straight to Puma
4	Puma	Checks your login session in Redis. Fetches page data from PostgreSQL
5	Puma → Gitaly	If the page shows code or diffs, Puma asks Gitaly for the file contents
6	Puma → Browser	Builds the HTML page and sends it back. Total time: under 500ms

You push code from your terminal

1	Terminal	Runs git push — connects to GitLab on port 22 (SSH)
2	GitLab Shell	Verifies your SSH key. Checks you have write access
3	Gitaly	Receives and stores the new commits, files, and branch update on disk
4	Puma	Notified of the push. Records the event in PostgreSQL
5	Redis queue	Puma drops jobs into Redis: start pipeline, send notifications, deliver webhooks
6	Sidekiq	Picks up jobs from Redis and executes them in the background

Component	Explain
HTTP/HTTPS	Your browser and API clients enter GitLab through this encrypted web port.
SSH	Developers push and pull code through this port using an SSH key — no password needed.
NGINX	First to receive web requests — handles encryption, security, and routing.
GitLab Shell	First to receive SSH requests — verifies your key and hands off to Gitaly.
Workhorse	Go service that handles heavy uploads and downloads so Ruby stays fast.
Puma	Web server that runs the Rails app — handles every real-time page load and API call.
Sidekiq	Also runs the Rails app, but for background tasks like pipelines, emails, and webhooks.
Redis	In-memory store for sessions, cache, and the Sidekiq job queue.
PostgreSQL	Permanent database storing all GitLab data — users, issues, CI records.
Gitaly	Git service that owns all repository storage — nothing bypasses it.

SECTION 1 — MANAGE

Manage

The Manage section controls who can access your project, how members are organized, and what members are allowed to do. It is the administrative backbone of a GitLab project.

Members

Members are users who have been granted access to the project. Each member is assigned a role that determines their permissions.

Role	What They Can Do
Minimal Access	View limited group information without access to projects. For more information, see Users with Minimal Access .
Guest	View and comment on issues and epics. Cannot push code or access repository. This role applies to private and internal projects only.
Planner	Create and manage issues, epics, milestones, and iterations. Focused on project planning and tracking with the ability to view and collaborate on code changes.
Reporter	View code, create issues, and generate reports. Cannot push code or manage protected branches.
Security Manager	View and manage security vulnerabilities, compliance configurations, and audit events. Focused on security operations without code push access.
Developer	Push code to non-protected branches, create merge requests, and run CI/CD pipelines. Cannot manage project settings.
Maintainer	Manage branches, merge requests, CI/CD settings, and project members. Cannot delete the project.
Owner	Full control over the project or group, including deletion and visibility settings.

Real-world example: A junior developer joins your team. You add them as a Developer so they can push feature branches but cannot modify protected main/production branches.

How to Add a Member

- Go to Manage > Members
- Click Invite members
- Enter the user's GitLab username or email
- Select the appropriate role
- Optionally set an expiration date (great for contractors)

Groups

Groups let you organize multiple projects and members under a single namespace. A group can contain sub-groups, creating a hierarchy that mirrors your organization.

Example: Company 'AcmeCorp' has a group. Inside it: sub-group 'Backend' (with api-service, auth-service repos) and sub-group 'Frontend' (with web-app, mobile-app repos). A Backend maintainer can access all backend repos without being added to each one individually.

Group-Level Permissions

- Members added to a group automatically inherit access to all projects in it.
- Group CI/CD variables are available to all pipelines in the group.
- Group-level runners can be shared across all projects.

SECTION 2 — PLAN

Plan

The Plan section is GitLab's built-in project management system. It replaces tools like Jira or Trello for many teams by providing issues, milestones, boards, and roadmaps directly alongside the codebase.

Issues

Issues are the fundamental unit of work. They represent tasks, bug reports, feature requests, or any item that needs tracking.

Issue Field	Purpose / Example
Title	Short description: 'Fix login timeout bug on mobile'
Description	Full context, reproduction steps, screenshots, acceptance criteria.
Assignee	Who is responsible for resolving this issue.
Labels	Categorize: 'bug', 'priority::high', 'backend'.
Milestone	Group issue to a sprint or release: 'Sprint 12 - Aug 2024'.
Due Date	Deadline for completion.
Weight	Effort estimation (story points equivalent): 1–10.
Linked Issues	Block/is-blocked-by relationships between issues.

Creating an Issue via GitLab Quick Actions

Inside an issue description or comment, you can use quick actions — slash commands that modify the issue:

```
/assign @john.doe  
/label ~bug ~priority::high  
/milestone %"Sprint 12"  
/estimate 4h  
/spend 2h
```

Example workflow: A QA engineer finds a bug. They create an issue titled 'Payment API returns 500 on empty cart'. They label it `~bug ~priority::critical`, assign it to the backend developer, and link it to the current sprint milestone. The developer sees it in their Kanban board and moves it to 'In Progress' when they start working.

Milestones

Milestones group issues and merge requests into a time-boxed deliverable — a sprint, a release, or a product phase.

- Set a start date and due date.
- Track completion percentage (closed issues / total issues).
- See all open issues and MRs assigned to the milestone.

Example: Milestone 'v2.0.0 Release - September 2024'. Attach all features and bug fixes planned for the v2.0 release. When 100% of issues close, you are ready to ship.

Boards

Boards provide a Kanban-style visualization of issues organized by their labels or status. Each column represents a stage in your workflow.

Default Columns

- Open — newly created issues
- In Progress — issues labeled `~In Progress`
- Review — issues in code review
- Closed — resolved issues

Power tip: Create a scoped label group called 'workflow' with values: `workflow::todo`, `workflow::in-progress`, `workflow::review`, `workflow::done`. Each board column maps to one value. Because labels are scoped, an issue can only be in one workflow stage at a time — exactly like a proper Kanban system.

Epics (Group Level)

Epics are high-level containers for multiple related issues, often spanning several milestones. They allow you to track large initiatives or product themes.

- Parent epics can contain child epics for multi-level planning.
- A roadmap view visualizes epics on a timeline.
- Health status indicators (on track / needs attention / at risk) give a quick project overview.

Example: Epic: 'Redesign Customer Checkout Flow'. Child issues: 'Update cart UI', 'Integrate Stripe v3', 'Add Apple Pay support', 'A/B test checkout page'. When all child issues close, the epic closes.

Iterations

Iterations are time-boxed cycles (like sprints in Scrum). You assign issues to iterations and track progress with a burndown chart.

Example: Set up 2-week iterations. Each iteration has a start date, end date, and a list of issues committed for that cycle. The burndown chart shows whether you are on track to complete all work by the end of the iteration.

SECTION 3 — AI (GitLab Duo)

AI — GitLab Duo

GitLab Duo is GitLab's suite of AI-powered features integrated into the platform. These features use large language models to assist developers at every stage of the software lifecycle.

Agents

GitLab Duo Agents are AI workflows that can autonomously perform multi-step tasks on your behalf. Examples include creating issues from a text description, summarizing discussions, or generating code based on acceptance criteria.

Agent Capability	What It Does
Issue summarization	Reads a long issue thread and produces a concise summary of decisions made.
Code explanation	Explains what a function or file does in plain English.
Vulnerability explanation	Describes a detected security vulnerability and how to fix it.
Root cause analysis	Analyzes failing CI pipeline logs and suggests the likely cause.
Test generation	Generates unit tests for selected code.

Sessions (Duo Chat)

Sessions refer to interactive AI chat sessions (GitLab Duo Chat). You can open a chat panel within the GitLab UI or IDE extension and ask questions about your codebase, pipeline, or project.

Example Prompts

- 'Explain what the function processPayment() does in checkout.js'
- 'Why is my pipeline failing on the test stage?'
- 'Suggest how to refactor this SQL query for better performance'
- 'What security issues might exist in this merge request?'

Duo Chat is context-aware — it understands the file you have open, the pipeline log you are viewing, or the merge request you are reviewing. It does not just give generic answers; it incorporates your actual project context.

SECTION 4 — CODE

Code

The Code section is where all source code management happens. It maps directly to Git operations — repositories, branches, commits, tags — with a rich web UI on top.

Repository

The repository is the heart of the project. It stores all your source code, configuration files, documentation, and history.

Key Repository Features

- File browser with syntax highlighting
- Inline editing — edit files directly in the browser without cloning
- Blame view — see who last modified each line and in which commit
- History view — full commit log per file
- Web IDE — full browser-based code editor (VS Code-like)

Example: A new developer joins. They browse the repository to understand the project structure before cloning. They spot a typo in README.md and fix it directly in the browser using the edit button — no local setup required.

Merge Requests (MRs)

Merge Requests are the primary mechanism for code review and collaboration. A developer creates a MR to propose merging a feature branch into another branch (usually main or develop).

MR Feature	Description
Diff view	Side-by-side or unified view of all changes.
Inline comments	Comment directly on a specific line of changed code.
Approval rules	Require N approvals before merge is allowed.
Draft MR	Mark as WIP/Draft to indicate not ready for review yet.

Auto-merge	Merge automatically when all conditions are met.
Squash commits	Combine all MR commits into one clean commit on merge.
Pipelines in MR	CI/CD pipeline runs on the MR branch and status shown inline.
Code quality	Static analysis diff shows quality changes vs. target branch.

MR Workflow Example

- Developer creates branch: `git checkout -b feature/user-profile-update`
- Pushes commits and opens MR targeting main
- Assigns reviewer and adds label ~'needs-review'
- Reviewer leaves inline comments on specific lines
- Developer addresses feedback and pushes new commits
- Reviewer approves; CI pipeline passes
- Maintainer merges with 'Squash commits' enabled

Branches

Branches let multiple developers work on different features or fixes simultaneously without interfering with each other.

Branch Protection

- Protected branches prevent unauthorized force pushes or deletions.
- You can require pipelines to pass before merging.
- You can require a minimum number of approvals.
- Specify who can push directly vs. only via MR.

```
# Common branching strategy (GitFlow simplified)
main          - production-ready code only
develop       - integration branch for features
feature/*     - individual feature work
hotfix/*      - urgent production fixes
release/*     - release preparation
```

Commits

Every saved change in Git is a commit. GitLab's commit view shows the full history, diffs, and allows GPG signature verification.

Best practice: Use conventional commits format for readable history. Example: 'feat(auth): add OAuth2 login support', 'fix(api): handle null response from payment gateway', 'chore(deps): update Node.js to v20'.

Tags

Tags mark specific commits as significant — usually a release version. Unlike branches, tags do not move as new commits are added.

```
# Create and push a tag
git tag -a v1.4.0 -m "Release v1.4.0 - OAuth feature"
git push origin v1.4.0
```

GitLab can automatically trigger a pipeline when a tag matching a pattern is pushed. This is commonly used to trigger a production deployment pipeline only on tags matching v.*.*.*

Repository Graph

The repository graph is a visual diagram showing branch relationships and merge history. It is useful for understanding complex branching strategies and identifying long-lived divergent branches.

Compare Revisions

Compare any two branches, tags, or commits to see the exact diff. Useful before opening a MR or to audit changes between releases.

Snippets

Snippets are small pieces of code or text you want to store and share, separate from a full repository. They can be public, internal, or private.

Example: Store a frequently-used Kubernetes deployment template as a snippet and share the link with your team. Or save a complex bash one-liner you use in multiple projects.

SECTION 5 — BUILD (CI/CD)

Build — CI/CD Pipelines

The Build section is where GitLab's Continuous Integration and Continuous Delivery (CI/CD) capabilities live. Every time code is pushed, pipelines can automatically build, test, and package your application.

Pipelines

A pipeline is a series of automated stages and jobs defined in a `.gitlab-ci.yml` file at the root of your repository. Each pipeline run is triggered by a Git event (push, MR, schedule, tag).

Pipeline Stages — Visual Flow

- Stage 1: build → Compile code, build Docker image
- Stage 2: test → Run unit tests, integration tests
- Stage 3: scan → SAST, dependency check, container scan
- Stage 4: deploy → Deploy to staging or production

Basic `.gitlab-ci.yml` Example

```
stages:
  - build
  - test
  - deploy

build-image:
  stage: build
  image: docker:24
  script:
    - docker build -t myapp:$CI_COMMIT_SHA .
    - docker push myapp:$CI_COMMIT_SHA

run-tests:
  stage: test
  image: node:20
  script:
    - npm ci
```

```
- npm test

deploy-staging:
  stage: deploy
  script:
    - kubectl set image deploy/myapp app=myapp:$CI_COMMIT_SHA
  environment:
    name: staging
  only:
    - main
```

Jobs

A job is a single unit of work within a pipeline stage. Multiple jobs in the same stage run in parallel (if enough runners are available). Jobs in later stages wait for all jobs in the previous stage to pass.

Job Keyword	Purpose
script	The shell commands to execute.
image	Docker image to run the job inside.
before_script	Commands to run before the main script (e.g., install dependencies).
after_script	Commands to run after the main script (cleanup, notifications).
artifacts	Files to save and pass to later stages.
cache	Directories to cache between pipeline runs (e.g., node_modules).
rules	Conditions that determine when the job runs.
needs	Declare job dependencies to create a DAG pipeline.
environment	Associate job with an environment (staging, production).
allow_failure	Allow job to fail without blocking the pipeline.

Pipeline Editor

The Pipeline Editor is a browser-based YAML editor with syntax validation, autocomplete, and a visual pipeline diagram. You can edit `.gitlab-ci.yml` directly in GitLab without using a text editor locally.

Key benefit: The editor validates your YAML before you commit, and the visual diagram shows you the stage/job flow so you catch logical errors immediately.

Pipeline Schedules

Pipeline schedules trigger a pipeline on a cron schedule — not just on pushes. Common uses include nightly builds, weekly security scans, and database backups.

```
# Cron expression examples
0 2 * * *      - Every day at 2:00 AM
0 9 * * 1      - Every Monday at 9:00 AM
0 */6 * * *    - Every 6 hours
```

Example: Schedule a weekly container vulnerability scan to run every Sunday at midnight. This catches new CVEs in your base images even when no code changes have been made.

Artifacts

Artifacts are files produced by a job that are saved by GitLab and made available for download or for use in subsequent jobs.

```
build-app:
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
    expire_in: 7 days
    reports:
      junit: test-results.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
```

Artifact reports like JUnit XML and coverage XML are parsed by GitLab to display test results and coverage percentages directly in the MR interface.

SECTION 6 — SECURE

Secure

The Secure section contains GitLab's built-in security scanning tools. These integrate directly into CI/CD pipelines and surface vulnerabilities in the merge request, before code reaches production.

Security Capabilities

Scanner	What It Detects
SAST (Static Application Security Testing)	Code vulnerabilities: SQL injection, XSS, hardcoded secrets, insecure functions.
DAST (Dynamic Application Security Testing)	Running application vulnerabilities by simulating attacks.
Secret Detection	Hardcoded API keys, tokens, passwords committed to the repo.
Dependency Scanning	Known CVEs in your npm, pip, Maven, etc. dependencies.
Container Scanning	CVEs in the OS packages inside your Docker images.
IaC Scanning	Misconfigurations in Terraform, CloudFormation, Kubernetes YAML.
License Compliance	Identifies open-source licenses in dependencies (GPL, MIT, etc.)
SBOM (Software Bill of Materials)	Complete inventory of all dependencies and their versions.

Adding SAST to a Pipeline

```
include:  
  - template: Security/SAST.gitlab-ci.yml  
  - template: Security/Secret-Detection.gitlab-ci.yml  
  - template: Security/Dependency-Scanning.gitlab-ci.yml
```

These three lines in your .gitlab-ci.yml add comprehensive security scanning. GitLab's templates handle all the scanner configuration automatically. Results appear as a Security tab in the Merge Request.

Audit Events

Audit Events is a chronological log of all security-relevant actions taken in your project or group. It is critical for compliance, incident investigation, and SOC 2/ISO 27001 audits.

Audit Event Type	Example
Member added/removed	User john@example.com added as Developer on 2024-08-15.
Permission change	john@example.com role changed from Developer to Maintainer.
Branch protection changed	Protected branch 'main' settings updated.
Deploy key added	New deploy key added for production server.
Pipeline secret added	CI/CD variable AWS_SECRET_KEY added.
2FA disabled	User disabled two-factor authentication.

Security Configuration

The Security Configuration page is a central dashboard where you can enable or disable individual security scanners, configure scan policies, and see which scanners are active in your project.

- Toggle each scanner on/off without manually editing `.gitlab-ci.yml`
- Set scan execution policies (require scans before merge)
- Set vulnerability approval policies (block MR if critical CVE found)

SECTION 7 — DEPLOY

Deploy

The Deploy section manages everything related to releasing your application — releases, feature flags, package registries, container registries, and static pages.

Releases

GitLab Releases are versioned snapshots of your software, associated with a Git tag. They bundle release notes, changelogs, and downloadable assets.

Creating a Release via .gitlab-ci.yml

```
release-job:
  stage: deploy
  image: registry.gitlab.com/gitlab-org/release-cli:latest
  rules:
    - if: $CI_COMMIT_TAG
  script:
    - echo 'Creating release for $CI_COMMIT_TAG'
  release:
    tag_name: '$CI_COMMIT_TAG'
    name: 'Release $CI_COMMIT_TAG'
    description: './CHANGELOG.md'
```

Feature Flags

Feature flags (also called feature toggles) let you deploy code to production but control which users see new functionality. You can enable a feature for 1% of users, for internal users, or for a specific user by ID — all without a new deployment.

Feature Flag Strategy	Use Case
All users	Enable feature for 100% of users.
Percent of users	Gradual rollout — e.g., 5% then 25% then 100%.
User IDs	Enable only for specific beta testers.
User list	Enable for named users in a list.

Gradual rollout

Increment percentage over time.

Using Feature Flags in Code (Node.js Example)

```
const { initialize } = require('unleash-client');
const client = initialize({
  url:
'https://gitlab.example.com/api/v4/feature_flags/unleash/PROJECT_ID',
  appName: 'production',
  instanceId: 'UNLEASH_INSTANCE_ID'
});

if (client.isEnabled('new-checkout-flow')) {
  return renderNewCheckout();
} else {
  return renderLegacyCheckout();
}
```

Package Registry

The Package Registry is a built-in repository for software packages. You can publish and consume packages from your pipelines without relying on public registries.

Supported Format	Example Use Case
npm	Private Node.js packages for your organization.
Maven/Gradle	Internal Java libraries shared across services.
PyPI	Private Python packages.
NuGet	.NET packages.
Helm	Private Helm charts for Kubernetes deployments.
Generic	Any binary — installers, compiled binaries, firmware.

Container Registry

The Container Registry stores Docker images built by your pipelines. Images are scoped to the project and accessible with GitLab credentials.

```
# In .gitlab-ci.yml
build:
```

```
script:
  - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD
    $CI_REGISTRY
  - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
  - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

`CI_REGISTRY_USER`, `CI_REGISTRY_PASSWORD`, and `CI_REGISTRY` are predefined GitLab CI variables — you do not need to set them manually. The registry URL is always `registry.gitlab.com/<namespace>/<project>`.

Model Registry

The Model Registry (GitLab MLOps feature) stores versioned machine learning models alongside their metrics and artifacts. It integrates with the ML experiment tracking workflow.

Pages

GitLab Pages hosts static websites directly from your repository. It is ideal for documentation, project websites, and reports.

```
pages:
  stage: deploy
  script:
    - npm run build
    - mv dist/ public/
  artifacts:
    paths:
      - public
  only:
    - main
```

GitLab Pages supports custom domains with automatic Let's Encrypt TLS. Common use: publish your project's documentation site (built with MkDocs, Hugo, or Docusaurus) on every merge to main.

SECTION 8 — OPERATE

Operate

The Operate section provides tools to manage and observe your deployed infrastructure — Kubernetes clusters, Terraform states, and cloud integrations.

Environments

Environments represent the places where your application is deployed — development, staging, production. GitLab tracks deployments to each environment, giving you a history of what version is deployed where.

Environment Feature	Benefit
Deployment history	See every deployment: who deployed, when, what commit.
Rollback	One-click rollback to any previous deployment.
Environment-scoped variables	Set different DATABASE_URL for staging vs production.
Protected environments	Require manual approval before deploying to production.
Stop environment	Spin down review apps when the MR closes.

Review Apps

Review Apps automatically deploy your branch to a temporary environment when a MR is created. Each MR gets its own URL. Reviewers can test the actual running application, not just read code diffs.

```
review-app:
  stage: deploy
  script:
    - deploy-to-k8s.sh $CI_COMMIT_REF_SLUG
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_COMMIT_REF_SLUG.review.myapp.com
    on_stop: stop-review-app
```

Kubernetes Clusters

GitLab's Kubernetes integration connects your GitLab project to one or more Kubernetes clusters. The GitLab Agent for Kubernetes (agentk) runs inside your cluster and communicates back to GitLab.

- Deploy to Kubernetes from pipelines using the agent connection
- Browse workloads (Pods, Deployments, Services) from the GitLab UI
- View logs from running pods
- GitOps workflows: sync cluster state with a Git repo

Installing the GitLab Agent

```
# Create agent config in your repo
.gitlab/agents/production-agent/config.yaml

# Register agent in GitLab UI: Operate > Kubernetes clusters > Connect a
cluster

# Install with Helm
helm repo add gitlab https://charts.gitlab.io
helm install gitlab-agent gitlab/gitlab-agent \
  --namespace gitlab-agent \
  --set config.token=<AGENT_TOKEN> \
  --set config.kasAddress=wss://kas.gitlab.com
```

Terraform States

GitLab provides a built-in backend for Terraform state files. Instead of using S3 or Terraform Cloud to store state, you can store it directly in GitLab — secured by GitLab's access controls.

```
# In your Terraform backend config
terraform {
  backend "http" {
    address =
    "https://gitlab.com/api/v4/projects/PROJECT_ID/terraform/state/STATE_NAME"
    lock_address =
    "https://gitlab.com/api/v4/projects/PROJECT_ID/terraform/state/STATE_NAME/
    lock"
    unlock_address =
    "https://gitlab.com/api/v4/projects/PROJECT_ID/terraform/state/STATE_NAME/
    lock"
    username = "your-username"
    password = "$GITLAB_ACCESS_TOKEN"
  }
}
```

Terraform Modules

The Terraform Module Registry lets you publish reusable Terraform modules as versioned packages within GitLab. Teams consume these modules in their own Terraform code without copy-pasting.

Example: The platform team publishes a module 'vpc-standard' v1.2.0 to the registry. Product teams reference it as: source = 'gitlab.com/acmecorp/infrastructure/vpc-standard' and always get the same battle-tested VPC configuration.

Google Cloud

GitLab's Google Cloud integration enables deploying directly to GCP services (Cloud Run, GKE, Cloud Functions) from pipelines without managing service account keys. It uses workload identity federation for keyless authentication.

SECTION 9 — MONITOR

Monitor

The Monitor section provides observability features — error tracking, incident management, and alerting — directly within GitLab, so developers stay in context when investigating production issues.

Error Tracking

GitLab Error Tracking integrates with Sentry (or GitLab's own error tracking) to surface application errors directly in the GitLab UI. When an error is detected, you can see the stack trace, affected users, and create a GitLab issue from it with one click.

Error Tracking Feature	Value
Error list	See all unresolved errors, frequency, and affected user count.
Stack traces	Full stack trace and context for each error occurrence.
Create issue	Open a GitLab issue pre-filled with error details.
Resolve/ignore	Mark errors as resolved or ignored from the UI.
Sentry integration	Works with existing Sentry setup — no data migration needed.

Alerts

Alerts surface from monitoring tools (Prometheus, Grafana, PagerDuty, Opsgenie) into GitLab. When a metric threshold is breached, an alert appears in GitLab and can be assigned to an on-call engineer.

Alert Lifecycle

- Alert fires from Prometheus (e.g., CPU > 90% for 5 minutes)
- Alert appears in GitLab Monitor > Alerts
- On-call engineer is paged and acknowledges the alert
- Engineer creates an incident from the alert
- Incident is resolved; alert is closed

Incidents

Incidents are a special type of issue designed for managing production outages. They have additional fields like severity, a timeline, and an automatic runbook link.

Incident Severity	Typical Meaning
S1 — Critical	Complete service outage. All hands. Revenue impact.
S2 — High	Major feature broken. Most users affected.
S3 — Medium	Partial degradation. Workaround available.
S4 — Low	Minor issue. Small subset of users. No revenue impact.

Service Desk

Service Desk turns your GitLab project into a support ticketing system. Customers email a dedicated GitLab address and their email automatically creates an issue. Replies in the issue are emailed back to the customer.

Example: support@yourcompany.com is your Service Desk address. A customer sends a bug report email. GitLab creates an issue, assigns it to the support team, and when the engineer comments on the issue, the comment is automatically emailed back to the customer — all without leaving GitLab.

SECTION 10 — ANALYZE

Analyze

The Analyze section provides data-driven insights into your project's health, team performance, and DevOps metrics. These help you identify bottlenecks, measure improvement, and report to stakeholders.

Value Stream Analytics

Value Stream Analytics measures how long it takes to move work from idea to production — the full value stream. It breaks the lifecycle into stages and shows average time in each stage.

Stage	Measured As
Plan	Time from issue created to first code committed.
Code	Time from first commit to MR opened.
Review	Time from MR opened to MR merged.
Staging	Time from MR merged to deployment to staging.
Production	Time from MR merged to deployment to production.
Total Lead Time	Sum of all stages — idea to production.

DORA Metrics are surfaced here: Deployment Frequency, Lead Time for Changes, Change Failure Rate, and Time to Restore Service. These four metrics (defined by Google's DevOps Research and Assessment team) are the industry standard for measuring DevOps performance.

Contributor Analytics

Contributor Analytics shows commit activity per contributor over time. Use it to understand team contribution patterns, identify who worked on what period, and recognize top contributors.

CI/CD Analytics

CI/CD Analytics tracks pipeline health metrics: pipeline success rate, duration trends, and job failure rates over time.

- Pipeline success rate: percentage of pipelines that passed vs failed.
- Duration trend: is your pipeline getting faster or slower over time?
- Failure breakdown: which jobs fail most often?

Example: You notice pipeline duration increased 40% over 3 months. CI/CD Analytics shows the 'run-integration-tests' job is the culprit. You investigate and find 50 new tests were added without parallelization. You split them into 5 parallel jobs and restore build time.

Repository Analytics

Repository Analytics shows code contribution patterns: commit frequency, lines added/removed, programming language breakdown, and branch count over time.

Model Experiments

Model Experiments (MLOps feature) tracks machine learning training runs, hyperparameters, and metrics. Each training run is logged as an experiment, enabling comparison across different model configurations.

Experiment Tracking Field	Example
Run name	bert-base-lr0.001-bs32-epoch5
Parameters	learning_rate: 0.001, batch_size: 32, epochs: 5
Metrics	accuracy: 0.923, f1: 0.918, val_loss: 0.241
Artifacts	model.pkl, tokenizer.json, training_curves.png
Source	Linked to the specific commit that produced this run.

SECTION 11 — SETTINGS

Settings

The Settings section is the administrative control panel for your GitLab project. Here you configure everything about how the project behaves, who can access it, and how it integrates with external tools.

General

General settings contain the most fundamental project configuration: name, visibility level, archiving, and dangerous operations.

General Setting	Description
Project name & path	Change the project name or URL slug.
Visibility	Private (invite only), Internal (all GitLab users), Public.
Project avatar	Logo shown next to the project name.
Project topics	Tags for discoverability (e.g., 'nodejs', 'kubernetes').
Archive project	Make read-only without deleting. Good for ended projects.
Transfer project	Move project to a different namespace/group.
Delete project	Permanently remove. Usually requires confirmation.

Service Accounts

Service accounts are machine users — non-human GitLab accounts used by automation, bots, and CI/CD processes. They can be granted project access and generate tokens without tying permissions to a human user's account.

Best practice: Use a service account for your production deploy pipeline instead of a personal access token tied to an engineer's account. If the engineer leaves, the deployment pipeline continues to work.

Integrations

Integrations connect GitLab to external tools. GitLab has 50+ built-in integrations.

Integration	What It Does
Slack	Notify a Slack channel when pipelines pass/fail, issues are created, MRs are merged.
Jira	Link GitLab commits and MRs to Jira issues. Transitions Jira status from commit messages.
Jenkins	Trigger Jenkins builds from GitLab events.
Prometheus	Pull metrics from Prometheus for display in GitLab environments.
PagerDuty	Send alerts to PagerDuty for on-call escalation.
GitHub	Mirror a GitLab repo to/from GitHub.
Google Chat	Notify Google Spaces on pipeline events.
Datadog	Send CI/CD metrics to Datadog for monitoring.

Webhooks

Webhooks send an HTTP POST request to a URL you specify whenever a specific event happens in GitLab. They are the foundation of custom automation and third-party integrations.

Webhook Event	Example Use Case
Push events	Trigger a custom build server on every push.
Merge request events	Notify Slack when a MR is opened or merged.
Pipeline events	Update a deployment dashboard when pipeline status changes.
Issue events	Sync GitLab issues to an external project management tool.
Tag push events	Trigger a release workflow in an external system.
Confidential issue events	Trigger security-sensitive workflows separately.

Example Webhook Payload (Push Event)

```
{
  "object_kind": "push",
  "event_name": "push",
  "before": "95790bf891e76fee5726",
```

```
{
  "after": "da1560886d4f094c3e6c",
  "ref": "refs/heads/main",
  "user_name": "John Smith",
  "commits": [{
    "id": "da1560886d4f094c3e6c",
    "message": "feat(auth): add OAuth2 login\n",
    "author": { "name": "John Smith", "email": "john@example.com" }
  }]
}
```

Access Tokens

Access Tokens are credentials for programmatic access to GitLab. They replace passwords in API calls, Git operations, and CI/CD scripts.

Token Type	Scope & Use
Personal Access Token (PAT)	Tied to your account. Full API access with selected scopes.
Project Access Token	Scoped to one project. Ideal for project-level automation.
Group Access Token	Scoped to a group and all its projects.
Deploy Token	Read-only. For pulling images from registry or cloning repos.
CI/CD Job Token	Automatically generated for each job. Short-lived.

Repository (Settings)

Repository settings control Git-specific behaviors: default branch, push rules, protected branches, and deploy keys.

Push Rules

- Reject commits with secrets (email if commit message contains 'password')
- Require signed commits (GPG or SSH signature verification)
- Reject unsigned tags
- Enforce commit message format (e.g., must match conventional commits pattern)
- Restrict file types (e.g., block .exe uploads)
- Restrict file size (e.g., reject files > 10MB)

Merge Requests (Settings)

Merge Request settings control the merge behavior for the entire project.

MR Setting	Description
Merge method	Merge commit / Squash / Rebase — which Git history strategy to use.
Squash commits	Always / Allow / Never / Require squashing on merge.
Delete source branch	Automatically delete feature branches after merge.
Merge checks	Require pipeline to succeed before merging.
Approval rules	N approvals required; specific groups must approve.
Code owners	CODEOWNERS file determines who must approve changes to specific paths.

CI/CD (Settings)

CI/CD settings configure pipeline behavior, runners, and variables for the project.

CI/CD Variables

Variables store secrets and configuration values that jobs reference in scripts. They are injected as environment variables at job runtime.

```
# Referencing a variable in .gitlab-ci.yml
deploy:
  script:
    - echo "Deploying to $ENVIRONMENT"
    - aws s3 sync dist/ s3://$S3_BUCKET --region $AWS_REGION
```

Variable Type	Behavior
Variable (plain)	Value is shown in logs unless masked.
Masked	Value is hidden in job logs. Must be Base64 safe.
Protected	Only available in protected branches and tags.
File	Value is written to a temp file; variable contains the file path.
Expanded	GitLab expands \$VAR_NAME references inside the value.

Runners

- Shared runners — provided by GitLab.com; available to all projects.
- Group runners — available to all projects in a group.
- Project-specific runners — registered only to this project.
- Instance runners — self-hosted GitLab only; available to all projects on the instance.

Packages and Registries (Settings)

Controls whether the Package Registry, Container Registry, and Terraform Module Registry are enabled for the project. Also configures cleanup policies for old images and packages to manage storage.

Container Registry Cleanup Policy

```
# Example: keep last 10 tags, delete images older than 90 days
cadence: 1month
keep_n: 10
older_than: 90d
name_regex_keep: main|v.*
```

Monitor (Settings)

Monitor settings configure how error tracking and alerting are set up for this project — including the Sentry DSN connection and alert integrations.

Usage Quotas

Usage Quotas shows how much of your GitLab storage, compute minutes, and transfer bandwidth this project has consumed versus its plan limits.

Quota Type	What It Tracks
Storage	Repository size, LFS objects, artifacts, registry images, wiki.
Compute minutes	Total CI/CD pipeline minutes used this month.
Transfer	Egress bandwidth from registry pulls.
Seats	Number of billable members consuming a license seat.

QUICK REFERENCE SUMMARY

Summary: GitLab Sidebar at a Glance

Section / Feature	One-Line Summary
Manage > Members	Control who can access and what they can do.
Plan > Issues	Track work items: bugs, features, tasks.
Plan > Boards	Kanban visualization of issue workflow stages.
Plan > Milestones	Group issues into sprints or releases.
Plan > Epics	High-level theme spanning multiple milestones.
AI > Agents	Autonomous AI workflows for summarization, code gen, test writing.
AI > Sessions	Interactive chat with context-aware AI about your code.
Code > Repository	Source code storage with blame, history, Web IDE.
Code > Merge Requests	Code review workflow with inline comments and approvals.
Code > Branches	Parallel development lines with protection rules.
Code > Tags	Immutable version markers for releases.
Build > Pipelines	Automated build/test/deploy defined in .gitlab-ci.yml.
Build > Jobs	Individual units of work within a pipeline.
Build > Artifacts	Files saved by jobs for download or use in later stages.
Secure > SAST/DAST	Code and runtime vulnerability scanning in CI/CD.
Secure > Audit Events	Compliance log of all access and configuration changes.
Deploy > Releases	Versioned software release bundles with changelogs.
Deploy > Feature Flags	Toggle features for subsets of users without redeploying.

Deploy > Container Registry	Built-in Docker image storage.
Deploy > Pages	Host static websites from your repo.
Operate > Environments	Track deployments and manage rollbacks per environment.
Operate > Kubernetes	Manage K8s clusters and GitOps workflows.
Operate > Terraform States	Store Terraform state files in GitLab.
Monitor > Error Tracking	Surface Sentry errors in GitLab with issue creation.
Monitor > Incidents	Production outage management with severity and timeline.
Monitor > Service Desk	Email-based customer support ticketing system.
Analyze > Value Stream	Measure lead time from idea to production (DORA metrics).
Analyze > CI/CD Analytics	Pipeline success rates and duration trends.
Settings > Integrations	Connect to Slack, Jira, Prometheus, and 50+ tools.
Settings > Webhooks	HTTP callbacks triggered by GitLab events.
Settings > Access Tokens	API and Git credentials for automation.
Settings > CI/CD Variables	Secrets and config injected into pipeline jobs.

DevOps Shack

devopsshack.com | @devopsshack | YouTube: DevOps Shack